LAPORAN TUGAS
PEMROGRAMAN BERORIENTASI OBJEK
SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS



Disusun Oleh:

AHMAD RIBBIY ALDI	2509116100
EKO SATRIO WIBOWO	2509116028
RASYA ADITYA RAMADANI	2509116082
ZEFRI AL RIZQULLAH	2509116084

Dosen Pengampu:

Ir. M Ibadurrahman Arrasyid Supriyanto, M.Kom.

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

2026

DAFTAR ISI

DAFTAR ISI ..... **I**

DAFTAR GAMBAR ..... **III**

BAB I PENDAHULUAN **1**

1.1. Deskripsi Studi Kasus **1**

1.2. Tujuan Program **1**

BAB II PEMBAHASAN **3**

2.1. Array 1 Dimensi **3**

2.2. Array 2 Dimensi **4**

2.3. Array 3 Dimensi **4**

2.4. ArrayList **5**

2.5. Stack **6**

2.6. Queue **6**

BAB III IMPLEMENTASI & PENJELASAN SOURCE CODE **8**

3.1. Struktur Project **8**

3.2. Implementasi Class Buku **8**

3.3. Implementasi Class Buku Transaksi **11**

3.4. Implementasi Class InputUtil **13**

3.5. Implementasi Class Perpustakaan Controller **16**

3.6. Implementasi Class Main **28**

BAB IV HASIL EKSEKUSI & PENGUJIAN **30**

4.1. Menu Utama **30**

4.2. Pengelolaan Data Buku **30**

4.3. Kategori Buku **33**

4.4. Lokasi Buku **33**

4.5. Antrean Mahasiswa.....34

4.6. Proses Peminjaman Buku.....35

4.7. Riwayat Transaksi.....36

4.8. Statistik Peminjaman.....37

BAB V PENUTUP 38

5.1. Kesimpulan 38

DAFTAR GAMBAR

Gambar 3.1 Struktur Project 8
Gambar 4.1 Menu Utama 30
Gambar 4.2 Tampilan Data Buku 30
Gambar 4.3 Menambah Data Buku 31
Gambar 4.4 Memperbarui Data Buku 32
Gambar 4.5 Menghapus Data Buku 32
Gambar 4.6 Tampilan Kategori Buku 33
Gambar 4.7 Tampilan Lokasi Buku 33
Gambar 4.8 Input Mahasiswa ke Antrean 34
Gambar 4.9 Antrian Mahasiswa 34
Gambar 4.10 Pengujian Urutan Pelayanan 35
Gambar 4.11 Proses Peminjaman Buku 35
Gambar 4.12 Tampilan Keseluruhan Riwayat Transaksi 36
Gambar 4.13 Tampilan Riwayat Transaksi Terakhir 36
Gambar 4.14 Membatalkan Transaksi Terakhir 36
Gambar 4.15 Tampilan Statistik Peminjaman 37

BAB I
PENDAHULUAN

1.1. Deskripsi Studi Kasus

Sistem Manajemen Perpustakaan Kampus merupakan program berbasis Java Command Line Interface yang dirancang untuk membantu proses pengelolaan sederhana pada perpustakaan kampus. Program menangani data kategori buku, data koleksi buku, lokasi penyimpanan buku, antrean mahasiswa, proses peminjaman, riwayat transaksi, dan statistik peminjaman.

Dalam kegiatan perpustakaan, data yang dikelola memiliki karakteristik yang berbeda. Kategori buku memiliki jumlah data yang tetap. Data buku dapat bertambah, berubah, dan dihapus. Lokasi buku memiliki struktur berdasarkan lantai, rak, dan posisi. Mahasiswa yang akan melakukan peminjaman perlu dilayani berdasarkan urutan kedatangan. Riwayat transaksi juga perlu disimpan agar transaksi terakhir dapat diketahui dan dibatalkan apabila terjadi kesalahan.

Berdasarkan kebutuhan tersebut, program menerapkan enam struktur data, yaitu Array 1 Dimensi, Array 2 Dimensi, Array 3 Dimensi, ArrayList, Stack, dan Queue. Setiap struktur data digunakan sesuai karakteristik data yang dikelola.

Array 1 Dimensi digunakan untuk menyimpan kategori buku. Array 2 Dimensi digunakan untuk menyimpan statistik jumlah peminjaman berdasarkan kategori dan hari. Array 3 Dimensi digunakan untuk merepresentasikan lokasi buku berdasarkan lantai, rak, dan posisi. ArrayList digunakan untuk mengelola data buku secara dinamis. Stack digunakan untuk menyimpan riwayat transaksi peminjaman dengan prinsip Last In First Out. Queue digunakan untuk mengelola antrean mahasiswa berdasarkan prinsip First In First Out.

Keenam struktur data tersebut diintegrasikan dalam proses peminjaman. Mahasiswa terlebih dahulu masuk ke antrean. Ketika mahasiswa dilayani, sistem memeriksa buku yang dipilih, stok, kategori, dan lokasinya. Apabila peminjaman berhasil, stok buku dikurangi, statistik peminjaman diperbarui, transaksi disimpan dalam Stack, dan mahasiswa dikeluarkan dari antrean.

1.2. Tujuan Program

Tujuan pembuatan Sistem Manajemen Perpustakaan Kampus adalah:

1. Menerapkan Array 1 Dimensi untuk menyimpan kategori buku.

2. Menerapkan Array 2 Dimensi untuk mengelola statistik peminjaman berdasarkan kategori dan hari.
3. Menerapkan Array 3 Dimensi untuk merepresentasikan lokasi penyimpanan buku.
4. Menerapkan ArrayList untuk mengelola data buku secara dinamis.
5. Menerapkan Stack untuk menyimpan dan membatalkan transaksi peminjaman terakhir.
6. Menerapkan Queue untuk mengelola antrean mahasiswa berdasarkan urutan kedatangan.
7. Mengintegrasikan keenam struktur data dalam satu alur pengelolaan perpustakaan.
8. Menerapkan konsep Pemrograman Berorientasi Objek melalui penggunaan class, object, constructor, atribut, method, encapsulation, dan pemisahan tanggung jawab antarclass.
9. Menerapkan validasi input agar data yang dimasukkan sesuai dengan kebutuhan sistem.

BAB II
PEMBAHASAN

2.1. Array 1 Dimensi

Array 1 Dimensi pada program digunakan untuk menyimpan daftar kategori buku yang tersedia di perpustakaan. Kategori tersebut terdiri dari Pemrograman, Matematika, Jaringan Komputer, dan Basis Data.

Data kategori dapat digambarkan sebagai berikut:

No	Kategori Buku
1	Pemrograman
2	Matematika
3	Jaringan Komputer
4	Basis Data

Array 1 Dimensi dipilih karena data kategori hanya berupa satu kelompok data yang tersusun secara berurutan. Jumlah kategori pada program juga telah ditentukan sehingga tidak memerlukan perubahan data secara dinamis selama program berjalan. Kategori tersebut digunakan sebagai acuan ketika petugas menambahkan atau memperbarui data buku. Petugas tidak perlu mengetik nama kategori secara bebas, melainkan memilih salah satu kategori yang telah tersedia. Sebagai contoh, buku “Pemrograman Java” menggunakan kategori Pemrograman, sedangkan buku “Matematika Diskrit” menggunakan kategori Matematika. Dengan cara tersebut, penggunaan kategori menjadi lebih konsisten. Program dapat menghindari perbedaan penulisan kategori seperti “Programming”, “Pemograman”, atau bentuk lainnya yang sebenarnya memiliki maksud yang sama.

Dalam alur program, kategori buku juga digunakan pada proses pencatatan statistik peminjaman. Ketika sebuah buku dipinjam, sistem mengetahui kategori buku tersebut sehingga jumlah peminjaman pada kategori yang sesuai dapat diperbarui. Jadi, fungsi utama Array 1 Dimensi dalam program adalah sebagai tempat penyimpanan kategori buku sekaligus sebagai acuan kategori yang digunakan oleh data buku dan statistik peminjaman.

2.2. Array 2 Dimensi

Array 2 Dimensi digunakan untuk menyimpan data jumlah peminjaman buku berdasarkan dua informasi, yaitu kategori buku dan hari dalam satu minggu.

Data tersebut dapat digambarkan dalam bentuk tabel berikut:

Data tersebut dapat digambarkan dalam bentuk tabel berikut:

Kategori	Sen	Sel	Rab	Kam	Jum	Sab	Min
Pemrograman	1	0	0	0	0	0	0
Matematika	0	0	0	0	0	0	0
Jaringan Komputer	0	0	0	0	0	0	0
Basis Data	0	0	0	0	0	0	0

Baris pada tabel menunjukkan kategori buku, sedangkan kolom menunjukkan hari peminjaman. Oleh karena itu, Array 2 Dimensi sesuai digunakan karena data yang disimpan memiliki hubungan antara dua bagian tersebut. Pada awal program, seluruh jumlah peminjaman bernilai 0. Nilai tersebut akan berubah secara otomatis ketika terjadi peminjaman. Sebagai contoh, mahasiswa bernama Andi Saputra meminjam buku “Pemrograman Java” pada hari Senin. Karena buku tersebut termasuk kategori Pemrograman, jumlah peminjaman kategori Pemrograman pada hari Senin bertambah dari 0 menjadi 1.

Apabila pada hari yang sama Budi Santoso meminjam buku “Jaringan Komputer Dasar”, maka statistik kategori Jaringan Komputer pada hari Senin juga bertambah menjadi 1. Dengan demikian, Array 2 Dimensi tidak hanya digunakan untuk memenuhi kebutuhan penyimpanan data, tetapi juga memberikan informasi sederhana mengenai aktivitas peminjaman buku berdasarkan kategori dan hari.

2.3. Array 3 Dimensi

Array 3 Dimensi digunakan untuk merepresentasikan lokasi penyimpanan buku di dalam perpustakaan. Lokasi buku pada program dibentuk berdasarkan tiga tingkatan, yaitu:

Lantai -> Rak -> Posisi Buku

Perpustakaan pada program memiliki 2 lantai. Setiap lantai memiliki 3 rak, dan setiap rak menyediakan 4 posisi penyimpanan buku.

Sebagai contoh, data awal buku memiliki lokasi:

ID	Judul Buku	Lantai	Rak	Posisi
B001	Pemrograman Java	1	1	1
B002	Matematika Diskrit	1	1	2
B003	Jaringan Komputer Dasar	1	2	1
B004	How to Expert at Database	2	1	1

Array 3 Dimensi sesuai digunakan karena lokasi buku tidak hanya memiliki satu atau dua informasi, tetapi terdiri dari tiga tingkatan lokasi. Penggunaan lokasi juga terhubung dengan pengelolaan data buku. Ketika buku baru ditambahkan, petugas menentukan lokasi penyimpanannya. Program kemudian memastikan lokasi tersebut belum digunakan oleh buku lain.

Misalnya Lantai 1, Rak 1, Posisi 1 sudah ditempati oleh B001. Jika petugas mencoba menempatkan buku baru pada lokasi yang sama, program akan menolak proses tersebut. Apabila lokasi buku diperbarui, lokasi lama akan dikosongkan dan lokasi baru akan digunakan. Dengan demikian, Array 3 Dimensi berfungsi sebagai representasi sederhana kondisi penyimpanan fisik buku pada perpustakaan.

2.4. ArrayList

ArrayList digunakan sebagai tempat penyimpanan utama data koleksi buku. Berbeda dengan kategori yang jumlahnya telah ditentukan, jumlah buku dalam perpustakaan dapat berubah. Buku baru dapat ditambahkan, data buku dapat diperbarui, dan buku tertentu dapat dihapus. Oleh karena itu, diperlukan struktur data yang lebih dinamis.

Tabel Ini merupakan penerapan ArrayList untuk menyimpan sebuah data sementara.

ID	JUDUL	KATEGORI	STOK	LOKASI
B001	Pemrograman Java	Pemrograman	5	Lantai 1, Rak 1, Posisi 1
B002	Matematika Diskrit	Matematika	4	Lantai 1, Rak 1, Posisi 2
B003	Jaringan Komputer Dasar	Jaringan Komputer	3	Lantai 1, Rak 2, Posisi 1

B004	How to Expert at Database	Basis Data	6	Lantai 2, Rak 1, Posisi 1

2.5. Stack

Stack digunakan untuk menyimpan riwayat transaksi peminjaman buku. Stack memiliki konsep Last In First Out (LIFO), yang berarti data yang terakhir dimasukkan merupakan data pertama yang dapat diambil kembali. Konsep tersebut digunakan pada program untuk menyediakan fitur melihat dan membatalkan peminjaman terakhir. Misalnya terdapat tiga transaksi dengan urutan:

URUTAN	MAHASISWA	BUKU
1	Zefri Al Rizqullah	B001 Pemrograman Java
2	Yahya Jailani	B002 Jaringan Komputer Dasar
3	Riaz	B003 Basis Data

Stack pada program memiliki fungsi penting ketika terjadi kesalahan pada peminjaman terakhir. Misalnya petugas salah memilih buku ketika melayani Citra. Sistem dapat membatalkan transaksi tersebut. Pembatalan bukan berarti proses pengembalian buku biasa. Pembatalan berfungsi sebagai proses undo terhadap transaksi peminjaman terakhir yang salah atau tidak jadi dilakukan.

2.6. Queue

Queue digunakan untuk mengelola antrean mahasiswa yang ingin melakukan peminjaman buku. Queue memiliki prinsip First In First Out (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang keluar atau dilayani. Konsep tersebut sesuai dengan sistem antrean pada perpustakaan karena mahasiswa yang datang terlebih dahulu seharusnya mendapatkan pelayanan terlebih dahulu. Misalnya mahasiswa masuk dengan urutan:

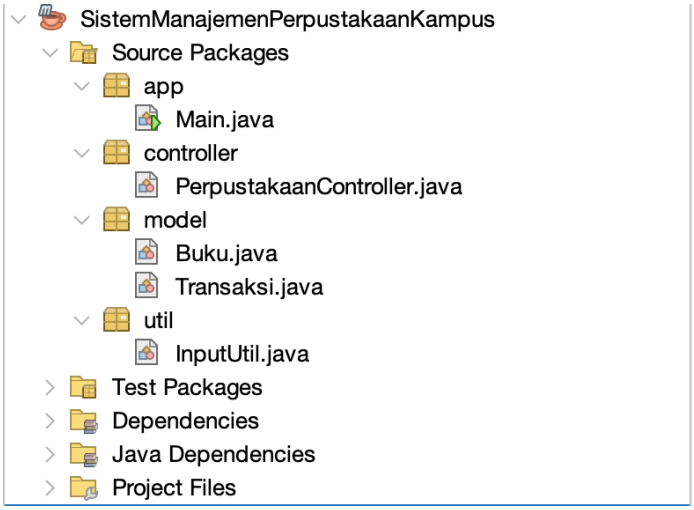
URUTAN	MAHASISWA
1	Zefri Al Rizqullah
2	Yahya Jailani
3	Riaz

Maka antreannya dimulai dari Zefri yang paling depan dan Riaz yang paling belakang. Zefri menjadi mahasiswa pertama yang dilayani dan ketika Zefri berhasil melakukan peminjaman, Zefri akan dikeluarkan dari antrean sehingga antrean mahasiswa selanjutnya yang berikutnya dilayani hingga seterusnya sampai antrean kosong.

BAB III
IMPLEMENTASI & PENJELASAN SOURCE CODE

3.1. Struktur Project

Project Sistem Manajemen Perpustakaan Kampus dibuat menggunakan bahasa pemrograman Java dan dijalankan dalam bentuk Command Line Interface (CLI). Program dibagi menjadi empat package utama, yaitu app, model, controller, dan util.



Gambar 3.1 Struktur Project

Pembagian package dilakukan berdasarkan fungsi masing-masing bagian program. Package app digunakan sebagai tempat class utama yang menjalankan program. Package model digunakan untuk merepresentasikan objek atau data yang digunakan dalam sistem. Package controller berisi proses utama dan logika pengolahan data perpustakaan. Sementara itu, package util berisi fungsi pendukung yang digunakan untuk menangani dan memvalidasi input pengguna.

3.2. Implementasi Class Buku

Class Buku digunakan untuk merepresentasikan sebuah buku di dalam sistem. Setiap object Buku menyimpan informasi mengenai ID, judul, kategori, stok, dan lokasi penyimpanan.

3.2.1. Deklarasi Class dan Atribut Buku

```
package model;
```

```
public class Buku {
    private String id;
    private String judul;
    private String kategori;
    private int stok;
    private int lantai;
    private int rak;
    private int posisi;
```

Baris "package model;" menunjukkan bahwa class Buku berada pada package model. Package tersebut digunakan untuk mengelompokkan class yang merepresentasikan data atau object dalam sistem. Dan baris "public class buku" mendeklarasikan sebuah class bernama Buku. Kata public menunjukkan bahwa class dapat digunakan oleh class lain dari package yang berbeda.

Selanjutnya terdapat tujuh atribut dan seluruh atribut menggunakan access modifier private. Hal tersebut merupakan penerapan encapsulation karena data tidak dapat diakses atau diubah secara langsung dari class lain.

3.2.2. Constructor Buku

```
public Buku(String id, String judul, String kategori, int stok,
            int lantai, int rak, int posisi) {
    this.id = id;
    this.judul = judul;
    this.kategori = kategori;
    this.stok = stok;
    this.lantai = lantai;
    this.rak = rak;
    this.posisi = posisi;
}
```

Kode tersebut merupakan constructor dari class Buku. Constructor digunakan untuk memberikan nilai awal ketika object Buku dibuat. Parameter id, judul, kategori, stok, lantai, rak, dan posisi menerima data yang diberikan saat pembuatan object. Perintah seperti this.id

= id digunakan untuk memasukkan nilai parameter id ke atribut id milik object. Hal yang sama dilakukan pada atribut lainnya. Sebagai contoh, ketika dibuat object new Buku("B001", "Pemrograman Java", "Pemrograman", 5, 1, 1, 1), maka program membuat buku dengan ID B001, judul Pemrograman Java, kategori Pemrograman, stok 5, dan lokasi Lantai 1, Rak 1, Posisi 1.

3.2.3. Getter & Setter

```
public String getId() { return id; }
public String getJudul() { return judul; }
public String getKategori() { return kategori; }
public int getStok() { return stok; }
public int getLantai() { return lantai; }
public int getRak() { return rak; }
public int getPosisi() { return posisi; }

public void setJudul(String judul) { this.judul = judul; }
public void setKategori(String kategori) { this.kategori = kategori; }
public void setStok(int stok) { this.stok = stok; }
public void setLantai(int lantai) { this.lantai = lantai; }
public void setRak(int rak) { this.rak = rak; }
public void setPosisi(int posisi) { this.posisi = posisi; }
```

Kode tersebut terdiri dari getter dan setter. Getter digunakan untuk mengambil nilai atribut Buku yang bersifat private, misalnya getId() digunakan untuk mendapatkan ID buku dan getStok() untuk mendapatkan jumlah stok. Sementara itu, setter digunakan untuk mengubah data buku, seperti setJudul() untuk mengubah judul, setKategori() untuk mengubah kategori, dan setStok() untuk mengubah jumlah stok. Setter juga digunakan untuk memperbarui lokasi melalui setLantai(), setRak(), dan setPosisi(). Method setter terutama digunakan pada fitur Update Buku sehingga data object yang sudah ada dapat diperbarui tanpa membuat object baru.

3.2.4. Method Pengelolaan Stok

```
public void kurangiStok() {
    if (stok > 0) {
        stok--;
    }
}

public void tambahStok() {
    stok++;
}
```

Method kurangiStok() digunakan ketika peminjaman berhasil. Sebelum stok dikurangi, percabangan if (stok > 0) memastikan bahwa buku masih tersedia sehingga stok tidak menjadi negatif. Perintah stok-- kemudian mengurangi stok sebanyak satu. Sebaliknya, tambahStok() digunakan ketika peminjaman terakhir dibatalkan dengan menambahkan stok sebanyak satu melalui stok++. Sebagai contoh, apabila B001 memiliki stok 5 dan berhasil dipinjam, stok menjadi 4. Jika peminjaman tersebut dibatalkan, stok kembali menjadi 5.

3.3. Implementasi Class Buku Transaksi

3.3.1. Deklarasi Atribut Transaksi

```
private String namaMahasiswa;
private String idBuku;
private String judulBuku;
private String kategori;
private int indeksKategori;
private int indeksHari;
private String hari;
```

Atribut pada class Transaksi digunakan untuk menyimpan informasi dari setiap peminjaman yang berhasil dilakukan. namaMahasiswa menyimpan nama peminjam, idBuku dan judulBuku menyimpan buku yang dipinjam, sedangkan kategori menyimpan kategori buku tersebut. hari menyimpan hari terjadinya peminjaman. Selain itu, indeksKategori dan indeksHari menyimpan posisi kategori dan hari pada Array 2 Dimensi statistik peminjaman.

Kedua indeks tersebut diperlukan ketika transaksi dibatalkan agar program mengetahui bagian statistik yang harus dikurangi kembali.

3.3.2. Constructor Transaksi

```
public Transaksi(String namaMahasiswa,
                 String idBuku,
                 String judulBuku,
                 String kategori,
                 int indeksKategori,
                 int indeksHari,
                 String hari) {
    this.namaMahasiswa = namaMahasiswa;
    this.idBuku = idBuku;
    this.judulBuku = judulBuku;
    this.kategori = kategori;
    this.indeksKategori = indeksKategori;
    this.indeksHari = indeksHari;
    this.hari = hari;
}
```

Constructor Transaksi digunakan untuk membuat object transaksi setelah proses peminjaman berhasil. Seluruh informasi peminjaman diterima melalui parameter kemudian disimpan ke masing-masing atribut menggunakan this. Sebagai contoh, ketika Andi Saputra berhasil meminjam B001 Pemrograman Java pada hari Senin, data mahasiswa, ID buku, judul, kategori, indeks kategori, indeks hari, dan nama hari disimpan menjadi satu object Transaksi. Object tersebut kemudian dimasukkan ke Stack sebagai riwayat peminjaman.

3.3.3. Getter Transaksi

```
public String getNamaMahasiswa() { return namaMahasiswa; }
public String getIdBuku() { return idBuku; }
public String getJudulBuku() { return judulBuku; }
public String getKategori() { return kategori; }
public int getIndeksKategori() { return indeksKategori; }
```

```
public int getIndeksHari() { return indeksHari; }
public String getHari() { return hari; }
```

Getter digunakan untuk mengambil kembali informasi yang tersimpan di dalam object Transaksi. Method seperti getNamaMahasiswa(), getIdBuku(), dan getJudulBuku() digunakan untuk mendapatkan informasi transaksi, sedangkan getIndeksKategori() dan getIndeksHari() digunakan ketika melakukan pembatalan transaksi. Melalui kedua indeks tersebut, program dapat mengetahui statistik kategori dan hari yang sebelumnya bertambah sehingga dapat dikurangi kembali saat peminjaman dibatalkan.

3.4. Implementasi Class InputUtil

3.4.1. Scanner dan Input Pengguna

```
private static final Scanner INPUT = new Scanner(System.in);
```

Kode tersebut membuat object Scanner bernama INPUT yang digunakan untuk menerima input pengguna melalui keyboard. System.in menunjukkan bahwa sumber input berasal dari console. Object Scanner dibuat satu kali dan digunakan kembali oleh berbagai method validasi sehingga program tidak perlu membuat Scanner baru setiap kali meminta input.

3.4.2. Validasi Nama Mahasiswa

```
public static String inputNama(String pesan) {
    while (true) {
        System.out.print(pesan);
        String nama = INPUT.nextLine().trim();

        if (nama.isEmpty()) {
            System.out.println("Nama tidak boleh kosong.");
            continue;
        }

        if (!nama.matches("[A-Za-z ]+")) {
            System.out.println(
```

```
                "Nama hanya boleh berisi huruf dan spasi."
            );
            continue;
        }

        if (nama.length() < 3) {
            System.out.println(
                "Nama minimal terdiri dari 3 karakter."
            );
            continue;
        }

        return nama;
    }
}
```

Method inputNama() digunakan untuk menerima sekaligus memvalidasi nama mahasiswa. while (true) membuat proses input terus diulang sampai pengguna memberikan nama yang benar. Input dibaca menggunakan nextLine() dan trim() digunakan untuk menghilangkan spasi pada awal dan akhir input. Percabangan pertama memeriksa apakah nama kosong. Percabangan berikutnya memastikan nama hanya terdiri dari huruf dan spasi sehingga input seperti Andi123 atau @Andi ditolak. Selanjutnya program memastikan panjang nama minimal tiga karakter. Setiap input yang tidak valid menggunakan continue agar proses kembali ke awal dan meminta input lagi. Jika seluruh pemeriksaan berhasil, return nama mengembalikan nama untuk digunakan oleh program.

3.4.3. Validasi Input Angka

```
public static int inputAngka(
        String pesan,
        int minimal,
        int maksimal) {

    while (true) {
```

```
        System.out.print(pesan);
        String data = INPUT.nextLine().trim();

        try {
            int angka = Integer.parseInt(data);

            if (angka < minimal || angka > maksimal) {
                System.out.println(
                    "Masukkan angka "
                    + minimal + " sampai " + maksimal + "."
                );
                continue;
            }

            return angka;

        } catch (NumberFormatException e) {
            System.out.println("Input harus berupa angka.");
        }
    }
}
```

Method inputAngka() digunakan untuk menerima input angka sekaligus menentukan batas nilai minimal dan maksimal. Input awalnya dibaca sebagai String kemudian Integer.parseInt(data) mencoba mengubahnya menjadi integer. Percabangan memeriksa apakah angka berada di luar batas yang diperbolehkan. Simbol || berarti atau, sehingga kondisi benar apabila angka lebih kecil dari nilai minimal atau lebih besar dari nilai maksimal. try-catch digunakan agar program tidak berhenti ketika pengguna memasukkan teks seperti abc. Jika terjadi kesalahan perubahan String menjadi integer, bagian catch menampilkan pesan bahwa input harus berupa angka dan pengguna diminta mengisi kembali.

3.4.4. Validasi ID Buku

```
public static String inputIdBuku(String pesan) {
    while (true) {
        System.out.print(pesan);

        String id = INPUT.nextLine()
                .trim()
                .toUpperCase();

        if (!id.matches("B\\d{3}")) {
            System.out.println(
                "Format ID harus B diikuti 3 angka. Contoh B001."
            );
            continue;
        }

        return id;
    }
}
```

Method inputIdBuku() digunakan untuk memastikan ID buku memiliki format yang sama. Input dibaca menggunakan nextLine(), kemudian trim() menghilangkan spasi berlebih dan toUpperCase() mengubah huruf menjadi kapital. Kondisi B\\d{3} menentukan bahwa ID harus diawali huruf B dan diikuti tepat tiga angka, seperti B001 atau B015. Jika format tidak sesuai, program menampilkan pesan kesalahan dan meminta input kembali. Apabila format sudah benar, ID dikembalikan menggunakan return id.

3.5. Implementasi Class Perpustakaan Controller

3.5.1. Deklarasi Struktur Data

```
private final String[] kategori = {
    "Pemrograman",
    "Matematika",
    "Jaringan Komputer",
    "Basis Data"
```

```
};

private final String[] hari = {
    "Senin",
    "Selasa",
    "Rabu",
    "Kamis",
    "Jumat",
    "Sabtu",
    "Minggu"
};

private final int[][] statistikPeminjaman =
        new int[kategori.length][hari.length];

private final String[][][] lokasiBuku =
        new String[2][3][4];

private final ArrayList<Buku> daftarBuku =
        new ArrayList<>();

private final Stack<Transaksi> riwayatTransaksi =
        new Stack<>();

private final Queue<String> antreanMahasiswa =
        new LinkedList<>();
```

Kode tersebut mendeklarasikan struktur data utama yang digunakan program. Array kategori menyimpan empat kategori buku, sedangkan Array hari menyimpan nama hari Senin sampai Minggu. statistikPeminjaman merupakan Array 2 Dimensi yang menggunakan kategori sebagai baris dan hari sebagai kolom. lokasiBuku merupakan Array 3 Dimensi dengan ukuran 2 lantai, 3 rak pada setiap lantai, dan 4 posisi pada setiap rak. daftarBuku merupakan ArrayList yang menyimpan object Buku secara dinamis. riwayatTransaksi merupakan Stack yang menyimpan transaksi peminjaman berdasarkan prinsip LIFO,

sedangkan antreanMahasiswa merupakan Queue yang menyimpan mahasiswa berdasarkan prinsip FIFO.

3.5.2. Inisialisasi Data Awal

```
public PerpustakaanController() {
    isiDataAwal();
}

private void isiDataAwal() {
    tambahDataAwal(
        new Buku("B001", "Pemrograman Java",
                "Pemrograman", 5, 1, 1, 1)
    );
    tambahDataAwal(
        new Buku("B002", "Matematika Diskrit",
                "Matematika", 4, 1, 1, 2)
    );
    tambahDataAwal(
        new Buku("B003", "Jaringan Komputer Dasar",
                "Jaringan Komputer", 3, 1, 2, 1)
    );
    tambahDataAwal(
        new Buku("B004", "Basis Data",
                "Basis Data", 6, 2, 1, 1)
    );
}
```

Constructor PerpustakaanController() langsung menjalankan isiDataAwal() ketika object controller dibuat. Method tersebut membuat empat object Buku sebagai data awal, yaitu B001 Pemrograman Java, B002 Matematika Diskrit, B003 Jaringan Komputer Dasar, dan B004 Basis Data. Setiap object memiliki kategori, stok, serta lokasi yang berbeda. Data tersebut kemudian dikirim ke method tambahDataAwal() agar dimasukkan ke ArrayList dan lokasi penyimpanannya dicatat pada Array 3 Dimensi.

```
private void tambahDataAwal(Buku buku) {
    daftarBuku.add(buku);
    lokasiBuku
        [buku.getLantai() - 1]
        [buku.getRak() - 1]
        [buku.getPosisi() - 1] = buku.getId();
}
```

Method tambahDataAwal() menerima object Buku, kemudian daftarBuku.add(buku) memasukkannya ke ArrayList. Setelah itu ID buku dimasukkan ke Array 3 Dimensi berdasarkan lantai, rak, dan posisi. Pengurangan -1 digunakan karena pengguna mengenal lokasi mulai dari nomor 1, sedangkan indeks Array Java dimulai dari 0. Dengan demikian, Lantai 1, Rak 1, Posisi 1 disimpan pada indeks [0][0][0].

3.5.3. Menampilkan dan Memilih Kategori

```
public void tampilKategori() {
    System.out.println("\n=== KATEGORI BUKU ===");
    for (int i = 0; i < kategori.length; i++) {
        System.out.println((i + 1) + ". " + kategori[i]);
    }
}
```

Method tampilKategori() menampilkan seluruh kategori buku menggunakan perulangan for. Variabel i dimulai dari 0 karena indeks Array dimulai dari 0. Kondisi i < kategori.length membuat perulangan terus berjalan selama masih terdapat kategori, sedangkan i++ memindahkan proses ke data berikutnya. kategori[i] mengambil kategori sesuai indeks dan i + 1 digunakan agar nomor yang ditampilkan kepada pengguna dimulai dari 1.

```
private int pilihKategori() {
    tampilKategori();
    int pilihan = InputUtil.inputAngka(
        "Pilih kategori: ", 1, kategori.length
    );
```

```
    return pilihan - 1;
}
```

Method pilihKategori() terlebih dahulu menampilkan kategori, kemudian pengguna memilih nomor kategori. Input dibatasi dari 1 sampai jumlah kategori. Nilai pilihan dikurangi satu sebelum dikembalikan karena nomor yang dilihat pengguna dimulai dari 1, sedangkan indeks Array dimulai dari 0.

3.5.4. Mencari Buku

```
private Buku cariBuku(String id) {
    for (Buku buku : daftarBuku) {
        if (buku.getId().equalsIgnoreCase(id)) {
            return buku;
        }
    }
    return null;
}
```

Method cariBuku() digunakan untuk mencari object Buku berdasarkan ID. Perulangan for membaca setiap buku yang terdapat pada ArrayList. ID setiap buku dibandingkan dengan ID yang dicari menggunakan equalsIgnoreCase() sehingga perbedaan huruf besar dan kecil tidak memengaruhi pencarian. Jika ditemukan, object Buku langsung dikembalikan menggunakan return buku. Apabila seluruh data telah diperiksa tetapi ID tidak ditemukan, method mengembalikan null.

3.5.5. Menambah Buku

```
String id = InputUtil.inputIdBuku("ID Buku: ");

if (cariBuku(id) != null) {
    System.out.println("ID buku sudah digunakan.");
    return;
}
```

```
String judul = InputUtil.inputTeks("Judul buku: ");
int indeksKategori = pilihKategori();
int stok = InputUtil.inputAngka("Stok buku: ", 0, 999);
```

Proses penambahan buku dimulai dengan meminta ID menggunakan validasi dari InputUtil. Program kemudian menggunakan cariBuku(id) untuk memeriksa apakah ID tersebut sudah digunakan. Jika hasil pencarian tidak null, berarti buku sudah ada dan proses dihentikan menggunakan return. Jika ID tersedia, program meminta judul, kategori, dan stok. Kategori dipilih dari kategori yang sudah tersedia, sedangkan stok hanya menerima angka antara 0 sampai 999.

```
int lantai = InputUtil.inputAngka("Lantai (1-2): ", 1, 2);
int rak = InputUtil.inputAngka("Rak (1-3): ", 1, 3);
int posisi = InputUtil.inputAngka("Posisi (1-4): ", 1, 4);

if (lokasiBuku[lantai - 1][rak - 1][posisi - 1] != null) {
    System.out.println("Lokasi tersebut sudah ditempati buku lain.");
    return;
}
```

Setelah data dasar diterima, pengguna menentukan lantai, rak, dan posisi buku. Nilai tersebut dibatasi sesuai ukuran Array 3 Dimensi. Program kemudian memeriksa lokasi yang dipilih. Jika isi lokasi tidak null, berarti lokasi telah ditempati oleh buku lain sehingga penambahan dihentikan. Hal ini mencegah dua buku menggunakan tempat penyimpanan yang sama.

```
Buku buku = new Buku(
    id, judul, kategori[indeksKategori],
    stok, lantai, rak, posisi
);

daftarBuku.add(buku);
lokasiBuku[lantai - 1][rak - 1][posisi - 1] = id;
```

Setelah seluruh data dinyatakan valid, program membuat object Buku baru menggunakan constructor. Object tersebut kemudian dimasukkan ke ArrayList

menggunakan add(). ID buku juga dimasukkan ke Array 3 Dimensi sesuai lokasi yang dipilih sehingga data buku dan lokasi penyimpanannya saling terhubung.

3.5.6. Update Buku

```
String id = InputUtil.inputIdBuku("Masukkan ID buku: ");
Buku buku = cariBuku(id);

if (buku == null) {
    System.out.println("Buku tidak ditemukan.");
    return;
}
```

Update dimulai dengan meminta ID buku yang akan diperbarui. Program mencari ID tersebut pada ArrayList. Jika buku == null, berarti data tidak ditemukan sehingga update dihentikan. Jika ditemukan, object Buku tersebut digunakan untuk menerima perubahan data.

```
lokasiBuku
    [buku.getLantai() - 1]
    [buku.getRak() - 1]
    [buku.getPosisi() - 1] = null;

buku.setJudul(judulBaru);
buku.setKategori(kategori[kategoriBaru]);
buku.setStok(stokBaru);
buku.setLantai(lantaiBaru);
buku.setRak(rakBaru);
buku.setPosisi(posisiBaru);

lokasiBuku[lantaiBaru - 1][rakBaru - 1][posisiBaru - 1]
        = buku.getId();
```

Sebelum lokasi diperbarui, lokasi lama pada Array 3 Dimensi dikosongkan dengan memberikan nilai null. Selanjutnya setter digunakan untuk mengubah judul, kategori, stok, lantai, rak, dan posisi pada object Buku. Setelah object selesai diperbarui, ID buku

dimasukkan ke lokasi baru pada Array 3 Dimensi. Dengan proses tersebut, perubahan data buku dan perubahan lokasi tetap sesuai.

3.5.7. Menghapus Buku

```
Buku buku = cariBuku(id);

if (buku == null) {
    System.out.println("Buku tidak ditemukan.");
    return;
}

lokasiBuku
    [buku.getLantai() - 1]
    [buku.getRak() - 1]
    [buku.getPosisi() - 1] = null;

daftarBuku.remove(buku);
```

Proses penghapusan terlebih dahulu mencari buku berdasarkan ID. Jika tidak ditemukan, proses dihentikan. Jika ditemukan, lokasi buku pada Array 3 Dimensi terlebih dahulu dikosongkan menggunakan nilai null. Setelah itu remove(buku) menghapus object Buku dari ArrayList. Dengan demikian, data buku dan lokasi penyimpanannya sama-sama dihapus.

3.5.8. Menampilkan Lokasi Buku

```
for (int lantai = 0; lantai < lokasiBuku.length; lantai++) {
    for (int rak = 0; rak < lokasiBuku[lantai].length; rak++) {
        for (int posisi = 0;
            posisi < lokasiBuku[lantai][rak].length;
            posisi++) {
```

Kode tersebut menggunakan tiga perulangan karena lokasi buku berbentuk Array 3 Dimensi. Perulangan pertama membaca setiap lantai, perulangan kedua membaca setiap rak

pada lantai tersebut, dan perulangan ketiga membaca setiap posisi pada rak. Dengan cara tersebut, program dapat memeriksa seluruh lokasi mulai dari Lantai 1, Rak 1, Posisi 1 sampai lokasi terakhir. Jika suatu lokasi memiliki ID buku, program menampilkan ID dan judulnya. Jika tidak memiliki data, lokasi ditampilkan sebagai kosong.

3.5.9. Pengelolaan Antrean Mahasiswa

```
String nama = InputUtil.inputNama("Nama mahasiswa: ");

for (String mahasiswa : antreanMahasiswa) {
    if (mahasiswa.equalsIgnoreCase(nama)) {
        System.out.println(
            "Mahasiswa tersebut sudah berada dalam antrean."
        );
        return;
    }
}

antreanMahasiswa.offer(nama);
```

Program meminta nama mahasiswa melalui inputNama() sehingga nama sudah melewati validasi. Selanjutnya perulangan membaca seluruh mahasiswa yang telah berada dalam Queue untuk mencegah nama yang sama masuk dua kali. Jika ditemukan nama yang sama, proses dihentikan. Jika belum terdaftar, offer(nama) memasukkan mahasiswa ke bagian belakang Queue. Dengan prinsip FIFO, mahasiswa yang masuk lebih dahulu akan dilayani lebih dahulu.

3.5.10. Proses Peminjaman Buku

```
if (antreanMahasiswa.isEmpty()) {
    System.out.println("Tidak ada mahasiswa dalam antrean.");
    return;
}

String mahasiswa = antreanMahasiswa.peek();
```

Sebelum peminjaman dilakukan, program memeriksa apakah Queue kosong menggunakan isEmpty(). Jika tidak terdapat mahasiswa, proses dihentikan. Jika terdapat antrean, peek() mengambil mahasiswa yang berada paling depan tanpa langsung menghapusnya. Mahasiswa belum dikeluarkan karena proses peminjaman masih dapat gagal.

```
Buku buku = cariBuku(idBuku);

if (buku == null) {
    System.out.println(
        "Buku tidak ditemukan. Mahasiswa tetap berada di antrean."
    );
    return;
}

if (buku.getStok() <= 0) {
    System.out.println(
        "Stok buku habis. Mahasiswa tetap berada di antrean."
    );
    return;
}
```

Program mencari buku berdasarkan ID. Jika hasil pencarian null, buku tidak tersedia dan transaksi dihentikan. Selanjutnya stok diperiksa menggunakan getStok(). Jika stok bernilai 0 atau kurang, peminjaman juga dihentikan. Karena poll() belum dijalankan, mahasiswa tetap berada pada Queue apabila salah satu pemeriksaan tersebut gagal.

```
int indeksHari = pilihHari();
int indeksKategori = cariIndeksKategori(buku.getKategori());

buku.kurangiStok();
statistikPeminjaman[indeksKategori][indeksHari]++;
```

Jika buku tersedia, pengguna memilih hari peminjaman dan program mencari posisi kategori buku. Setelah seluruh data valid, kurangiStok() mengurangi stok buku sebanyak satu. Kemudian nilai pada Array 2 Dimensi ditambah satu berdasarkan kategori dan hari

peminjaman. Misalnya B001 kategori Pemrograman dipinjam pada Senin, maka statistik Pemrograman pada kolom Senin bertambah satu.

```
Transaksi transaksi =
    new Transaksi(
        mahasiswa,
        buku.getId(),
        buku.getJudul(),
        buku.getKategori(),
        indeksKategori,
        indeksHari,
        hari[indeksHari]
    );

riwayatTransaksi.push(transaksi);
antreanMahasiswa.poll();
```

Setelah peminjaman berhasil, program membuat object Transaksi yang berisi mahasiswa, buku, kategori, serta hari peminjaman. Object tersebut dimasukkan ke Stack menggunakan push(). Setelah semua proses berhasil, poll() mengeluarkan mahasiswa dari bagian depan Queue. Urutan tersebut penting karena mahasiswa baru dihapus dari antrean setelah peminjaman benar-benar berhasil.

3.5.11. Riwayat dan Pembatalan Peminjaman

```
if (riwayatTransaksi.isEmpty()) {
    System.out.println(
        "Tidak ada transaksi yang dapat dibatalkan."
    );
    return;
}

Transaksi transaksi = riwayatTransaksi.pop();
Buku buku = cariBuku(transaksi.getIdBuku());
```

Program terlebih dahulu memeriksa apakah Stack memiliki transaksi. Jika kosong, tidak ada peminjaman yang dapat dibatalkan. Jika terdapat data, pop() mengambil sekaligus menghapus transaksi yang berada paling atas. Karena Stack menerapkan LIFO, transaksi yang diambil merupakan peminjaman yang terakhir dilakukan. Setelah transaksi diperoleh, program mencari buku berdasarkan ID yang tersimpan pada transaksi tersebut.

```
if (buku != null) {
    buku.tambahStok();
}

int kategoriIndex = transaksi.getIndeksKategori();
int hariIndex = transaksi.getIndeksHari();

if (statistikPeminjaman[kategoriIndex][hariIndex] > 0) {
    statistikPeminjaman[kategoriIndex][hariIndex]--;
}
```

Jika buku masih ditemukan, tambahStok() mengembalikan satu stok yang sebelumnya berkurang saat peminjaman. Selanjutnya program mengambil indeks kategori dan hari dari object Transaksi. Statistik pada kategori dan hari tersebut kemudian dikurangi satu apabila nilainya masih lebih dari 0. Dengan demikian, pembatalan tidak hanya menghapus transaksi dari Stack, tetapi juga mengembalikan stok dan statistik ke kondisi sebelum peminjaman dilakukan.

3.5.12. Menampilkan Statistik Peminjaman

```
for (int i = 0; i < statistikPeminjaman.length; i++) {
    System.out.printf("%-20s", kategori[i]);

    for (int j = 0; j < statistikPeminjaman[i].length; j++) {
        System.out.printf(
            "%10d",
            statistikPeminjaman[i][j]
        );
    }
```

```
    System.out.println();
}
```

Kode tersebut menggunakan dua perulangan untuk membaca Array 2 Dimensi. Perulangan pertama menggunakan variabel i untuk membaca setiap kategori, sedangkan perulangan kedua menggunakan j untuk membaca seluruh hari pada kategori tersebut. Nilai statistikPeminjaman[i][j] menunjukkan jumlah peminjaman berdasarkan kombinasi kategori dan hari. printf() digunakan agar data ditampilkan dalam bentuk tabel yang lebih rapi.

3.6. Implementasi Class Main

3.6.1. Method Main dan Object Controller

```
public static void main(String[] args) {

    PerpustakaanController perpustakaan =
        new PerpustakaanController();

    int pilihan;
```

Method main() merupakan titik awal ketika program Java dijalankan. Di dalamnya dibuat object PerpustakaanController bernama perpustakaan. Object tersebut digunakan untuk mengakses seluruh fitur utama sistem. Pada saat object dibuat, constructor PerpustakaanController juga menjalankan proses pengisian data awal. Variabel pilihan bertipe integer digunakan untuk menyimpan nomor menu yang dipilih pengguna.

3.6.2. Menu Utama

```
do {
    System.out.println("1. Kelola Data Buku");
    System.out.println("2. Lihat Kategori Buku");
    System.out.println("3. Lihat Lokasi Buku");
    System.out.println("4. Antrean dan Peminjaman");
    System.out.println("5. Riwayat Transaksi");
    System.out.println("6. Statistik Peminjaman");
```

```
    System.out.println("0. Keluar");

    pilihan = InputUtil.inputAngka(
        "Pilih menu: ", 0, 6
    );
```

Menu utama ditempatkan di dalam perulangan do-while agar terus ditampilkan selama pengguna belum memilih keluar. System.out.println() digunakan untuk menampilkan setiap pilihan menu. Setelah menu ditampilkan, InputUtil.inputAngka() meminta pilihan pengguna dengan batas nilai 0 sampai 6. Validasi tersebut memastikan pengguna tidak dapat memasukkan menu di luar pilihan yang tersedia atau memasukkan data nonnumerik.

3.6.3. Percabangan Menu

```
switch (pilihan) {
    case 1: perpustakaan.menuBuku(); break;
    case 2: perpustakaan.tampilKategori(); break;
    case 3: perpustakaan.tampilLokasiBuku(); break;
    case 4: perpustakaan.menuAntrean(); break;
    case 5: perpustakaan.menuTransaksi(); break;
    case 6: perpustakaan.tampilStatistik(); break;
    case 0:
        System.out.println(
            "Program selesai. Terima kasih."
        );
        break;
}
```

Percabangan switch digunakan untuk menentukan fitur yang dijalankan berdasarkan nomor yang dipilih. Pilihan 1 membuka pengelolaan buku, pilihan 2 menampilkan kategori, pilihan 3 menampilkan lokasi buku, pilihan 4 membuka antrean dan peminjaman, pilihan 5 membuka riwayat transaksi, dan pilihan 6 menampilkan statistik. Pilihan 0 menampilkan pesan bahwa program selesai. Perintah break digunakan agar program berhenti pada case yang sesuai dan tidak melanjutkan ke pilihan lainnya.

BAB IV
HASIL EKSEKUSI & PENGUJIAN

4.1. Menu Utama

Menu utama merupakan tampilan pertama ketika program dijalankan. Menu ini menyediakan akses ke seluruh fitur Sistem Manajemen Perpustakaan Kampus.

```
========================================
  SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS
========================================
1. Kelola Data Buku
2. Lihat Kategori Buku
3. Lihat Lokasi Buku
4. Antrean dan Peminjaman
5. Riwayat Transaksi
6. Statistik Peminjaman
0. Keluar
Pilih menu:
```

Gambar 4.1 Menu Utama

Gambar tersebut menunjukkan tampilan menu utama yang menyediakan enam fitur, yaitu pengelolaan data buku, kategori buku, lokasi penyimpanan, antrean dan peminjaman, riwayat transaksi, serta statistik peminjaman. Pengguna dapat memilih fitur dengan memasukkan nomor menu yang tersedia. Menu utama menggunakan perulangan sehingga akan terus ditampilkan sampai pengguna memilih opsi keluar.

4.2. Pengelolaan Data Buku

Pengujian ini dilakukan untuk memastikan ArrayList dapat digunakan dalam proses pengelolaan data buku melalui operasi Create, Read, Update, dan Delete (CRUD).

4.2.1 Menampilkan Data Buku

```
=== KELOLA BUKU ===
1. Tampilkan Buku
2. Tambah Buku
3. Update Buku
4. Hapus Buku
0. Kembali
Pilih menu: 1

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
```

Gambar 4.2 Tampilan Data Buku

Hasil tersebut menunjukkan hasil penampilan data buku yang telah disiapkan ketika program dijalankan. Setiap buku memiliki ID, judul, kategori, stok, dan lokasi penyimpanan. Data tersebut disimpan menggunakan ArrayList sehingga dapat ditampilkan dan digunakan dalam proses pengelolaan maupun peminjaman buku.

4.2.2 Menambahkan Data Buku

```
=== TAMBAH BUKU ===
ID Buku: B005
Judul buku: Tereliye

=== KATEGORI BUKU ===
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
Pilih kategori: 2
Stok buku: 12

Tentukan lokasi buku.
Lantai (1-2): 2
Rak (1-3): 2
Posisi (1-4): 5
Masukkan angka 1 sampai 4.
Posisi (1-4): 4
Buku berhasil ditambahkan.

=== KELOLA BUKU ===
1. Tampilkan Buku
2. Tambah Buku
3. Update Buku
4. Hapus Buku
0. Kembali
Pilih menu: 1

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
B005 | Tereliye | Matematika | Stok: 12 | Lantai 2, Rak 2, Posisi 4
```

Gambar 4.3 Menambah Data Buku

Hasil tersebut menunjukkan proses penambahan buku baru dengan ID B005. Setelah data dimasukkan dan dinyatakan valid, program membuat object Buku dan menyimpannya ke dalam ArrayList. Program juga mencatat lokasi buku pada Array 3 Dimensi. Hasil penambahan dapat dibuktikan melalui daftar buku yang menampilkan B005 sebagai data baru.

4.2.3 Memperbarui Data Buku

```
=== UPDATE BUKU ===

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
B005 | Tereliye | Matematika | Stok: 12 | Lantai 2, Rak 2, Posisi 4
Masukkan ID buku: B005

Data saat ini:
B005 | Tereliye | Matematika | Stok: 12 | Lantai 2, Rak 2, Posisi 4
Judul baru: Terelite Vol 4

=== KATEGORI BUKU ===
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
Pilih kategori: 4
Stok baru: 15

Lokasi baru:
Lantai (1–2): 2
Rak (1–3): 1
Posisi (1–4): 1
Lokasi sudah ditempati buku lain.

=== KELOLA BUKU ===
1. Tampilkan Buku
2. Tambah Buku
3. Update Buku
4. Hapus Buku
0. Kembali
Pilih menu: 1

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
B005 | Tereliye | Matematika | Stok: 12 | Lantai 2, Rak 2, Posisi 4
```

Gambar 4.4 Memperbarui Data Buku

Hasil tersebut menunjukkan bahwa fitur Update dapat memperbarui data buku yang telah tersimpan. Perubahan dilakukan pada judul, stok, dan posisi penyimpanan B005. Program memperbarui object Buku yang sudah ada sekaligus mengosongkan lokasi lama dan mengisi lokasi baru pada Array 3 Dimensi. Dengan demikian, perubahan data buku dan lokasi penyimpanannya tetap sesuai.

4.2.4 Menghapus Data Buku

```
=== HAPUS BUKU ===

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
B005 | Tereliye | Matematika | Stok: 12 | Lantai 2, Rak 2, Posisi 4
Masukkan ID buku: B005
Buku berhasil dihapus.

=== KELOLA BUKU ===
1. Tampilkan Buku
2. Tambah Buku
3. Update Buku
4. Hapus Buku
0. Kembali
Pilih menu: 1

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
```

Gambar 4.5 Menghapus Data Buku

Hasil tersebut menunjukkan proses penghapusan data buku berdasarkan ID. Ketika B005 dihapus, program mengeluarkan object Buku dari ArrayList dan mengosongkan lokasi penyimpanannya pada Array 3 Dimensi. Hasil penghapusan dibuktikan dengan tidak ditemukannya B005 pada daftar buku setelah proses selesai.

4.3. Kategori Buku

```
=== KATEGORI BUKU ===
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
```

Gambar 4.6 Tampilan Kategori Buku

Hasil tersebut menunjukkan empat kategori buku yang disimpan menggunakan Array 1 Dimensi. Kategori ditampilkan secara berurutan dan digunakan sebagai pilihan ketika menambahkan atau memperbarui buku. Dengan menggunakan kategori yang telah ditentukan, program dapat menjaga konsistensi pengelompokan buku.

4.4. Lokasi Buku

```
=== LOKASI PENYIMPANAN BUKU ===
Lantai 1 | Rak 1 | Posisi 1 | B001 - Pemrograman Java
Lantai 1 | Rak 1 | Posisi 2 | B002 - Matematika Diskrit
Lantai 1 | Rak 1 | Posisi 3 | KOSONG
Lantai 1 | Rak 1 | Posisi 4 | KOSONG
Lantai 1 | Rak 2 | Posisi 1 | B003 - Jaringan Komputer Dasar
Lantai 1 | Rak 2 | Posisi 2 | KOSONG
Lantai 1 | Rak 2 | Posisi 3 | KOSONG
Lantai 1 | Rak 2 | Posisi 4 | KOSONG
Lantai 1 | Rak 3 | Posisi 1 | KOSONG
Lantai 1 | Rak 3 | Posisi 2 | KOSONG
Lantai 1 | Rak 3 | Posisi 3 | KOSONG
Lantai 1 | Rak 3 | Posisi 4 | KOSONG
Lantai 2 | Rak 1 | Posisi 1 | B004 - Basis Data
Lantai 2 | Rak 1 | Posisi 2 | KOSONG
Lantai 2 | Rak 1 | Posisi 3 | KOSONG
Lantai 2 | Rak 1 | Posisi 4 | KOSONG
Lantai 2 | Rak 2 | Posisi 1 | KOSONG
Lantai 2 | Rak 2 | Posisi 2 | KOSONG
Lantai 2 | Rak 2 | Posisi 3 | KOSONG
Lantai 2 | Rak 2 | Posisi 4 | KOSONG
Lantai 2 | Rak 3 | Posisi 1 | KOSONG
Lantai 2 | Rak 3 | Posisi 2 | KOSONG
Lantai 2 | Rak 3 | Posisi 3 | KOSONG
Lantai 2 | Rak 3 | Posisi 4 | KOSONG
```

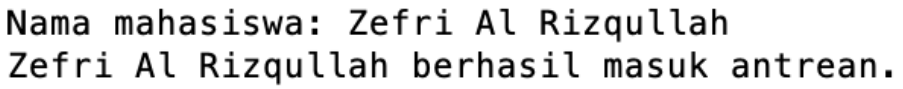
Gambar 4.7 Tampilan Lokasi Buku

Hasil tersebut menunjukkan lokasi penyimpanan buku menggunakan Array 3 Dimensi. Setiap lokasi dibedakan berdasarkan lantai, rak, dan posisi. Lokasi yang ditempati menampilkan ID dan judul buku, sedangkan lokasi yang belum digunakan ditampilkan dengan keterangan KOSONG. Hasil ini menunjukkan bahwa program dapat merepresentasikan lokasi buku berdasarkan tiga tingkatan penyimpanan.

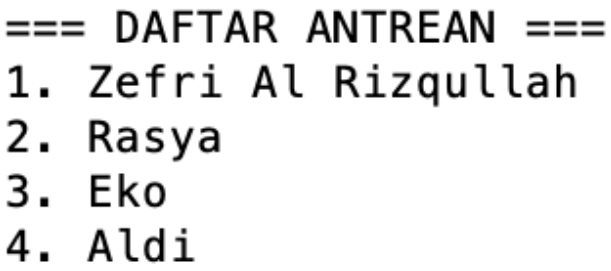
4.5. Antrean Mahasiswa

Pengujian ini bertujuan untuk memastikan antrean mahasiswa berjalan berdasarkan prinsip First In First Out (FIFO).

4.5.1 Menambahkan Mahasiswa ke Antrean



Gambar 4.8 Input Mahasiswa ke Antrean



Gambar 4.9 Antrian Mahasiswa

Hasil tersebut menunjukkan 4 mahasiswa yang berhasil dimasukkan ke dalam Queue berdasarkan urutan kedatangan. Zefri Al Rizqullah berada pada urutan pertama, diikuti oleh data mahasiswa lainnya jika ditambahkan lagi. Urutan tersebut menunjukkan penerapan prinsip FIFO, yaitu mahasiswa yang masuk terlebih dahulu akan mendapatkan pelayanan terlebih dahulu.

4.5.2 Menguji Urutan Pelayanan

```
Peminjaman berhasil.
Rasya meminjam Matematika Diskrit
Sisa stok: 3

=== ANTREAN PEMINJAMAN ===
1. Tambah Mahasiswa ke Antrean
2. Tampilkan Antrean
3. Layani Mahasiswa
0. Kembali
Pilih menu: 2

=== DAFTAR ANTREAN ===
1. Eko
2. Aldi
```

Gambar 4.10 Pengujian Urutan Pelayanan

Hasil tersebut menunjukkan perubahan antrean setelah Rasya berhasil melakukan peminjaman. Rasya dikeluarkan dari Queue sehingga Eko menjadi mahasiswa pertama yang menunggu pelayanan. Hasil tersebut membuktikan bahwa Queue menerapkan prinsip FIFO karena mahasiswa yang lebih dahulu masuk akan lebih dahulu dilayani.

4.6. Proses Peminjaman Buku

```
Mahasiswa yang sedang dilayani: Zefri Al Rizqullah

=== DAFTAR BUKU ===
B001 | Pemrograman Java | Pemrograman | Stok: 5 | Lantai 1, Rak 1, Posisi 1
B002 | Matematika Diskrit | Matematika | Stok: 4 | Lantai 1, Rak 1, Posisi 2
B003 | Jaringan Komputer Dasar | Jaringan Komputer | Stok: 3 | Lantai 1, Rak 2, Posisi 1
B004 | Basis Data | Basis Data | Stok: 6 | Lantai 2, Rak 1, Posisi 1
Masukkan ID buku yang dipinjam: B001

=== PILIH HARI ===
1. Senin
2. Selasa
3. Rabu
4. Kamis
5. Jumat
6. Sabtu
7. Minggu
Pilih hari: 2

Lokasi buku:
Lantai 1, Rak 1, Posisi 1

Peminjaman berhasil.
Zefri Al Rizqullah meminjam Pemrograman Java
Sisa stok: 4
```

Gambar 4.11 Proses Peminjaman Buku

Hasil tersebut menunjukkan bahwa Zefri Al Rizqullah berhasil meminjam buku B001 Pemrograman Java pada hari Selasa. Program memeriksa ketersediaan buku dan stok sebelum memproses peminjaman. Setelah transaksi berhasil, stok B001 berkurang dari 5 menjadi 4, statistik

peminjaman kategori Pemrograman pada hari Selasa bertambah satu, dan transaksi disimpan ke Stack. Mahasiswa kemudian dikeluarkan dari Queue karena proses pelayanan telah selesai.

4.7. Riwayat Transaksi

Pengujian dilakukan untuk memastikan Stack menyimpan transaksi berdasarkan prinsip Last In First Out (LIFO).

4.7.1 Menampilkan Riwayat Transaksi

```
=== RIWAYAT TRANSAKSI ===
Rasya | B002 | Matematika Diskrit | Matematika | Senin
Zefri Al Rizqullah | B001 | Pemrograman Java | Pemrograman | Selasa
```

Gambar 4.12 Tampilan Keseluruhan Riwayat Transaksi

Hasil tersebut menunjukkan riwayat transaksi peminjaman yang disimpan menggunakan Stack. Transaksi Rasya berada pada urutan paling atas karena merupakan transaksi yang terakhir dilakukan. Selanjutnya terdapat transaksi Zefri Al Rizqullah. Urutan tersebut menunjukkan penerapan prinsip LIFO, yaitu transaksi terakhir ditempatkan pada bagian teratas Stack.

4.7.2 Melihat Transaksi Terakhir

```
Transaksi terakhir:
Rasya | B002 | Matematika Diskrit | Matematika | Senin
```

Gambar 4.13 Tampilan Riwayat Transaksi Terakhir

Hasil tersebut menunjukkan hasil penggunaan operasi peek() untuk melihat transaksi terakhir pada Stack. Program menampilkan transaksi Rasya sebagai transaksi terbaru tanpa menghapusnya dari riwayat. Dengan demikian, pengguna dapat mengetahui peminjaman terakhir yang dilakukan.

4.7.3 Membatalkan Transaksi Terakhir

```
Transaksi berhasil dibatalkan.
Rasya | B002 | Matematika Diskrit | Matematika | Senin
Stok buku dikembalikan menjadi: 4
```

Gambar 4.14 Membatalkan Transaksi Terakhir

Hasil tersebut menunjukkan proses pembatalan peminjaman terakhir menggunakan Stack. Program mengambil transaksi Rasya dari bagian teratas Stack dan menghapusnya dari riwayat. Setelah pembatalan, stok B002 dikembalikan dari 3 menjadi 4 dan statistik peminjaman kategori Basis Data pada hari Senin dikurangi satu. Proses ini menunjukkan bahwa Stack digunakan untuk membatalkan transaksi terakhir sekaligus mengembalikan perubahan data yang terjadi akibat peminjaman.

4.8. Statistik Peminjaman

```
=== STATISTIK PEMINJAMAN ===
Kategori                  Sen       Sel       Rab       Kam       Jum       Sab       Min
Pemrograman                 0         1         0         0         0         0         0
Matematika                  0         0         0         0         0         0         0
Jaringan Komputer           0         0         0         0         0         0         0
Basis Data                  0         0         0         0         0         0         0
```

Gambar 4.15 Tampilan Statistik Peminjaman

Hasil tersebut menunjukkan hasil pencatatan statistik peminjaman menggunakan Array 2 Dimensi. Statistik mencatat satu peminjaman kategori Pemrograman pada hari Selasa. Sementara itu, statistik Matematika pada hari Senin kembali bernilai 0 karena transaksi Rasya telah dibatalkan. Hasil tersebut menunjukkan bahwa statistik dapat bertambah ketika peminjaman berhasil dan berkurang ketika transaksi terakhir dibatalkan.

BAB V
PENUTUP

5.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang dilakukan, proyek Sistem Manajemen Perpustakaan Kampus merupakan program berbasis Java yang dirancang untuk membantu pengelolaan data buku, kategori, lokasi penyimpanan, antrean mahasiswa, peminjaman, riwayat transaksi, dan statistik peminjaman. Program ini menerapkan konsep Pemrograman Berorientasi Objek (PBO) serta mengintegrasikan enam struktur data, yaitu Array 1 Dimensi, Array 2 Dimensi, Array 3 Dimensi, ArrayList, Stack, dan Queue.

Setiap struktur data memiliki fungsi yang berbeda dan saling mendukung dalam proses pengelolaan perpustakaan. Array 1 Dimensi digunakan untuk menyimpan kategori buku, Array 2 Dimensi untuk mencatat statistik peminjaman berdasarkan kategori dan hari, serta Array 3 Dimensi untuk mengatur lokasi buku berdasarkan lantai, rak, dan posisi. Sementara itu, ArrayList digunakan untuk mengelola data buku secara dinamis melalui fitur CRUD, Stack digunakan untuk menyimpan riwayat transaksi dan membatalkan peminjaman terakhir berdasarkan prinsip LIFO, sedangkan Queue digunakan untuk mengatur antrean mahasiswa berdasarkan prinsip FIFO.

Penerapan keenam struktur data tersebut memungkinkan berbagai fitur program saling terhubung. Ketika mahasiswa melakukan peminjaman, sistem memproses antrean, memeriksa ketersediaan buku, mengurangi stok, memperbarui statistik, serta menyimpan transaksi. Program juga dilengkapi validasi input untuk mencegah kesalahan, seperti penggunaan ID buku yang sama, pemilihan lokasi yang sudah ditempati, dan peminjaman buku yang tidak tersedia.

Melalui proyek ini, dapat dipahami bahwa pemilihan struktur data yang tepat berperan penting dalam pengelolaan dan pengolahan informasi. Selain memperdalam pemahaman mengenai penggunaan enam struktur data, proyek ini juga memberikan pengalaman dalam menerapkan class, object, constructor, encapsulation, method, percabangan, perulangan, serta pengelolaan input pada bahasa pemrograman Java.

Secara keseluruhan, Sistem Manajemen Perpustakaan Kampus telah dirancang dengan mengintegrasikan keenam struktur data sesuai kebutuhan studi kasus perpustakaan. Program ini menjadi contoh penerapan struktur data dalam sebuah sistem yang memiliki proses saling

berkaitan. Meskipun masih berbasis Command Line Interface (CLI) dan menggunakan penyimpanan sementara di memori, program dapat menjadi dasar untuk pengembangan sistem perpustakaan yang lebih lengkap dengan penambahan database, antarmuka grafis, serta fitur pengembalian buku pada masa mendatang.